GCA - Report 3

Maarten Roedolf

February 2024

1 Max on CPU

```
int findMaxCPu(int *input, int size)
{
    int max = input[0];

    for (int i = 1; i < size; i++)
    {
        if (input[i] > max)
        {
            max = input[i];
        }
    }

    return max;
}
```

Figure 1: Max element on CPU

2 Max with atomicMax

```
__global__ void findMaxAtomic(int *array, int *max, int size)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    __shared__ int local_max;
    local_max = array[0];

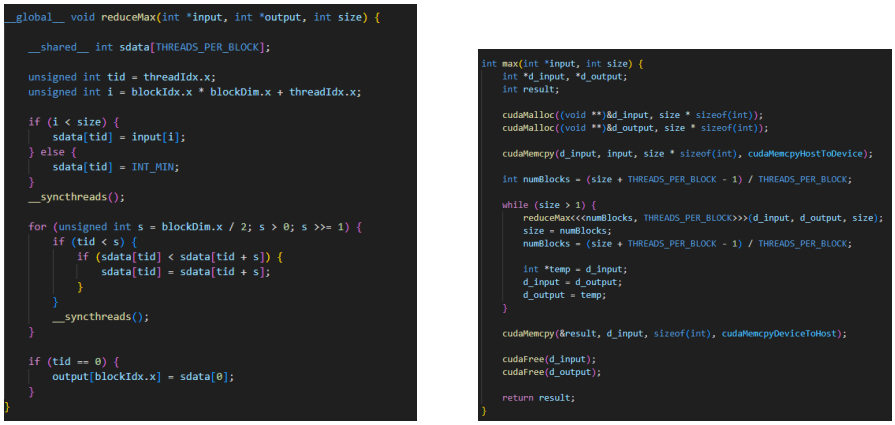
    for (int i = tid; i < size; i += stride)
    {
        atomicMax(&local_max, array[i]);
    }
    atomicMax(max, local_max);
}
```

Figure 2: Max element with atomicMax

Each thread in a block processes a portion of the array, updating a shared local_max variable atomically. Finally, the maximum value found within the block is stored globally using another atomic operation. I have added striding to ensure all elements are processed, even if there's more elements than threads.



3 Max with reduction



```cuda
__global__ void reduceMax(int *input, int *output, int size) {

    __shared__ int sdata[THREADS_PER_BLOCK];

    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < size) {
        sdata[tid] = input[i];
    } else {
        sdata[tid] = INT_MIN;
    }
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (sdata[tid] < sdata[tid + s]) {
                sdata[tid] = sdata[tid + s];
            }
        }
        __syncthreads();
    }

    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}
```

```cuda
int max(int *input, int size) {
    int *d_input, *d_output;
    int result;

    cudaMalloc((void **)&d_input, size * sizeof(int));
    cudaMalloc((void **)&d_output, size * sizeof(int));

    cudaMemcpy(d_input, input, size * sizeof(int), cudaMemcpyHostToDevice);

    int numBlocks = (size + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;

    while (size > 1) {
        reduceMax<<<numBlocks, THREADS_PER_BLOCK>>>(d_input, d_output, size);
        size = numBlocks;
        numBlocks = (size + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;

        int *temp = d_input;
        d_input = d_output;
        d_output = temp;
    }

    cudaMemcpy(&result, d_input, sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(d_input);
    cudaFree(d_output);

    return result;
}
```

Figure 3: Code for Max Element using Reduction

The reduceMax kernel operates by loading elements from the input array into shared memory within each block. It then performs a reduction within each block to find the maximum value. This reduction process iteratively compares pairs of elements within the shared memory array, storing the maximum of each pair back into the array until only one maximum value remains.

The maxGPU function manages the GPU-based reduction process by repeatedly calling the reduceMax kernel until the final maximum value is obtained. It handles memory allocation, data transfer between the host and the device, and cleanup. I dynamically adjust the grid and block dimensions based on the input size to ensure all elements are covered. After obtaining the final maximum value, it copies the result back to the host and frees the allocated memory on the device before returning the result.

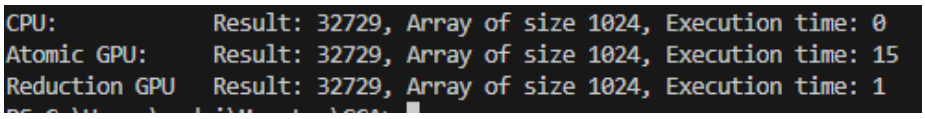


Figure 4: Results

4 Questions

How does time increase as a function of input size?:

Generally, as the input size increases, the execution time for both CPU and GPU methods will also increase. The time increase on the cpu is linear. However, the method using atomic max starts of pretty slow but gets pretty effecient while increasing the array. The reduction method on the GPU also stays really effeciënt, especially when the array size is a power of 2.

Calling Kernel with Fewer Threads Than Data Elements:

I have implemented striding to ensure al elements in the array always get processed.

Tackling a 2D Array:

We can define the grid dimensions in 2D (blockIdx.x, blockIdx.y, threadIdx.x, and threadIdx.y) and use these dimensions to calculate the x and y coordinates in our array. The base of our calculations would be the same as in a 1D array.